

LAPORAN PRAKTIKUM STRUKTUR DATA
PEKAN 6 TENTANG DOUBLE LINKED LIST



Disusun Oleh :

M.FAJAR FADHILUL ZIKRI

NIM 2311533025

Dosen Pengampu :

DR. WAHYUDI, S.T, M.T

Asisten Praktikum :

SHERLY SUKMADIRA PUTRI

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, TAHUN 2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa atas limpahan rahmat dan karunia-Nya, sehingga laporan praktikum ini dapat diselesaikan tepat pada waktunya. Laporan ini disusun sebagai dokumentasi dan evaluasi dari materi mata kuliah Struktur Data, yang berfokus pada cara mengorganisasi serta mengelola data secara logis dalam sistem komputer.

Praktikum ini memberikan pemahaman mengenai pentingnya pemilihan metode penyimpanan data yang tepat. Struktur data bukan sekadar cara menyimpan informasi, melainkan fondasi dalam membangun algoritma yang efisien. Dengan pengorganisasian data yang baik, sebuah program dapat memproses informasi dengan kecepatan yang lebih tinggi dan penggunaan memori yang lebih optimal.

Melalui rangkaian praktikum ini, mahasiswa diharapkan tidak hanya menguasai teori pengolahan data, tetapi juga mampu mengimplementasikannya dalam bentuk kode program yang sistematis. Kemampuan untuk menyusun data secara terstruktur merupakan keterampilan krusial yang dibutuhkan untuk memecahkan masalah komputasi yang lebih kompleks di masa mendatang.

Terima kasih kami sampaikan kepada dosen pembimbing atas bimbingan dan ilmu yang diberikan selama proses pembelajaran. Terima kasih juga kepada rekan-rekan mahasiswa yang telah memberikan dukungan serta berbagi ide selama praktikum berlangsung. Penyusun menyadari bahwa laporan ini masih memiliki ruang untuk perbaikan, sehingga kritik dan saran yang membangun akan sangat dihargai.

Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca dan membantu dalam memahami betapa pentingnya peran struktur data dalam dunia pengembangan perangkat lunak.

Padang, 2026

Penyusun

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB 1 PENDAHULUAN.....	1
1.1 Pengertian Praktikum.....	1
1.2 Tujuan Praktikum.....	1
1.3 Persyaratan Praktikum.....	1
1.4 Tempat dan Waktu Praktikum.....	2
BAB 2 PEMBAHASAN PRAKTIKUM.....	3
2.1 Pengertian DLL.....	3
2.2 Langkah-Langkah Praktikum.....	3
BAB 3 PENUTUP.....	15
3.1 Kesimpulan.....	15
LAMPIRAN.....	16

BAB 1

PENDAHULUAN

1.1 Pengertian Praktikum

Praktikum Struktur Data adalah kegiatan pembelajaran yang dilakukan di laboratorium komputer untuk mengasah keterampilan mahasiswa dalam memahami serta menerapkan cara pengorganisasian, penyimpanan, dan pengelolaan data secara efisien.

Kegiatan ini tidak hanya menekankan pada penguasaan teori mengenai model-model data, tetapi juga pada latihan implementasi struktur penyimpanan yang optimal, analisis efisiensi algoritma, hingga pengujian performa program. Praktikum dipandang sebagai wahana latihan yang menjembatani pemahaman konseptual mengenai tata kelola data dengan kemampuan teknis dalam memecahkan masalah komputasi yang kompleks secara sistematis.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum ini antara lain sebagai berikut :

1. Membantu mahasiswa memahami konsep dasar struktur data melalui penerapan langsung.
2. Melatih kemampuan menulis, mengompilasi, dan mengeksekusi program dengan mengikuti aturan sintaksis Struktur data.
3. Meningkatkan keterampilan dalam memecahkan masalah (problem solving) dengan pendekatan algoritmik.
4. Membiasakan mahasiswa bekerja sistematis dalam menyusun laporan yang memuat analisis hasil praktikum.
5. Menanamkan sikap teliti, disiplin, serta tanggung jawab dalam melaksanakan kegiatan laboratorium.
6. Mengetahui dan mengaplikasikan Tipe Data stack, queue dll.

1.3 Persyaratan Praktikum

Agar praktikum berjalan lancar, mahasiswa perlu memenuhi beberapa persyaratan berikut:

1. Telah mengikuti perkuliahan teori Struktur Data sebagai dasar pemahaman.
2. Membawa perlengkapan yang diperlukan, antara lain laptop atau komputer yang sudah terpasang Java Development Kit (JDK) dan Integrated Development Environment (IDE) yang direkomendasikan.
3. Mengikuti setiap sesi praktikum sesuai jadwal yang ditetapkan dan hadir minimal sesuai ketentuan program studi.
4. Mematuhi tata tertib laboratorium, termasuk menjaga keamanan data, perangkat, serta lingkungan kerja.
5. Menyusun laporan praktikum dengan format dan aturan yang telah ditetapkan dalam pedoman ini.

1.4 Waktu dan Tempat Praktikum

Pelaksanaan praktikum struktur data mengikuti kalender akademik yang berlaku pada program studi. Setiap sesi praktikum dilaksanakan sesuai jadwal yang ditentukan oleh dosen pengampu. Tempat kegiatan umumnya berlangsung di laboratorium komputer, namun pada kondisi tertentu dapat dilaksanakan secara mandiri dengan perangkat masing-masing, selama memenuhi syarat teknis yang ditetapkan.

BAB 2

PEMBAHASAN PRAKTIKUM

2.1 Pengertian DLL

Double Linked List (DLL) atau Senarai Berantai Ganda adalah struktur data linier di mana setiap elemen (disebut **node**) memiliki dua referensi atau *pointer*. Berbeda dengan Single Linked List yang hanya bisa berjalan satu arah, DLL memungkinkan kita untuk melintasi daftar baik ke depan maupun ke belakang.

1. Komponen Utama Node

Setiap node dalam Double Linked List terdiri dari tiga bagian utama:

1. **Data:** Menyimpan nilai atau informasi (misalnya integer, string, atau objek).
2. **Next Pointer:** Menyimpan alamat memori dari node selanjutnya.
3. **Prev Pointer:** Menyimpan alamat memori dari node sebelumnya.

Selain itu, terdapat dua pointer khusus untuk mengelola list:

- **Head:** Menunjuk ke node pertama dalam list.
- **Tail:** Menunjuk ke node terakhir dalam list.

2. Karakteristik Double Linked List

- **Dua Arah:** Navigasi dapat dilakukan dari head ke tail (forward) maupun dari tail ke head (backward).
- **Fleksibilitas Penghapusan:** Menghapus node lebih efisien karena kita tidak perlu mencari node sebelumnya secara manual; node tersebut sudah tersimpan di pointer prev.
- **Memori:** Membutuhkan lebih banyak memori dibandingkan Single Linked List karena adanya penyimpanan tambahan untuk pointer prev.

3. Operasi Dasar

Beberapa operasi standar yang sering dilakukan pada DLL meliputi:

- **Traversal:** Mengunjungi setiap node. Bisa dilakukan secara *forward* (menggunakan next) atau *backward* (menggunakan prev).
- **Insertion (Penambahan):**
 - Di awal (*Insert First*).
 - Di akhir (*Insert Last*).
 - Di tengah/setelah node tertentu.
- **Deletion (Penghapusan):**
 - Menghapus node pertama atau terakhir.
 - Menghapus node dengan nilai tertentu.

2.7 Langkah Langkah Praktikum

Pada praktikum kali ini kita akan membuat 4 buah class yang pertama Adalah **NodeDLL_2511533023**:

```
public class NodeDLL_2511533023 {  
    // mendefinisikan class node  
    int data_3023; //data  
    NodeDLL_2511533023 next_3023; // pointer ke next node  
    NodeDLL_2511533023 prev_3023; // pointer previous node  
  
    // konstruktor  
    public NodeDLL_2511533023(int data) {  
        this.data_3023 = data;  
        this.next_3023 = null;  
        this.prev_3023 = null;  
    }  
}
```

Hasil Output:

Tidak ada karena ini merupakan class blueprint atau kerangka saja yang nanti hasil nya dapat di panggil pakai class driver.

Class ke-2 adalah InsertDLL_2511533023:

```
public class InsertDLL_2511533023 {  
    // menambah node di awal DLL  
    static NodeDLL_2511533023 insertBegin_3023(NodeDLL_2511533023 head_3023, int  
data_3023) {  
        // buat node baru  
        NodeDLL_2511533023 new_node_3023 = new NodeDLL_2511533023(data_3023);  
        // jadikan pointer nextnya head  
        new_node_3023.next_3023 = head_3023;  
        // jadikan pointer prev head ke new_node  
        if (head_3023 != null) {  
            head_3023.prev_3023 = new_node_3023;  
        }  
        return new_node_3023;  
    }  
  
    // fungsi menambahkan node di akhir  
    public static NodeDLL_2511533023 insertEnd_3023(NodeDLL_2511533023 head_3023,  
int newData_3023) {  
        // buat node baru  
        NodeDLL_2511533023 newNode_3023 = new NodeDLL_2511533023(newData_3023);  
        // jika dll null jadikan head  
        if (head_3023 == null) {  
            head_3023 = newNode_3023;  
        } else {  
            NodeDLL_2511533023 curr_3023 = head_3023;  
            while (curr_3023.next_3023 != null) {  
                curr_3023 = curr_3023.next_3023;  
            }  
            curr_3023.next_3023 = newNode_3023;  
            newNode_3023.prev_3023 = curr_3023;  
        }  
    }  
}
```

```

        return head_3023;
    }

    // fungsi menambahkan node di posisi tertentu
    public static NodeDLL_2511533023 insertAtPosition_3023(NodeDLL_2511533023
head_3023, int pos_3023, int new_data_3023) {
    // buat node baru
    NodeDLL_2511533023 new_node_3023 = new NodeDLL_2511533023(new_data_3023);
    if (pos_3023 == 1) {
        new_node_3023.next_3023 = head_3023;
        if (head_3023 != null) {
            head_3023.prev_3023 = new_node_3023;
        }
        head_3023 = new_node_3023;
        return head_3023;
    }
    NodeDLL_2511533023 curr_3023 = head_3023;
    for (int i_3023 = 1; i_3023 < pos_3023 - 1 && curr_3023 != null; ++i_3023)
{
        curr_3023 = curr_3023.next_3023;
    }
    if (curr_3023 == null) {
        System.out.println("Posisi tidak ada");
        return head_3023;
    }
    new_node_3023.prev_3023 = curr_3023;
    new_node_3023.next_3023 = curr_3023.next_3023;
    curr_3023.next_3023 = new_node_3023;
    if (new_node_3023.next_3023 != null) {
        new_node_3023.next_3023.prev_3023 = new_node_3023;
    }
    return head_3023;
}

public static void printList_3023(NodeDLL_2511533023 head_3023) {
    NodeDLL_2511533023 curr_3023 = head_3023;
    while (curr_3023 != null) {
        System.out.print(curr_3023.data_3023 + " <-> ");
        curr_3023 = curr_3023.next_3023;
    }
    System.out.println();
}

public static void main(String[] args) {
    // membuat dll 2 <-> 3 <-> 5
    NodeDLL_2511533023 head_3023 = new NodeDLL_2511533023(2);
    head_3023.next_3023 = new NodeDLL_2511533023(3);
    head_3023.next_3023.prev_3023 = head_3023;
    head_3023.next_3023.next_3023 = new NodeDLL_2511533023(5);
    head_3023.next_3023.next_3023.prev_3023 = head_3023.next_3023;

    // cetak DLL awal
    System.out.print("DLL Awal: ");
    printList_3023(head_3023);

    // tambah 1 di awal
    head_3023 = insertBegin_3023(head_3023, 1);
    System.out.print("simpul 1 ditambah di awal: ");
    printList_3023(head_3023);
}

```



```

// tambah 6 di akhir
System.out.print("simpul 6 ditambah di akhir: ");
int data_3023 = 6;
head_3023 = insertEnd_3023(head_3023, data_3023);
printList_3023(head_3023);

// menambahkan node 4 di posisi 4
System.out.print("tambah node 4 di posisi 4: ");
int data2_3023 = 4;
int pos_3023 = 4;
head_3023 = insertAtPosition_3023(head_3023, pos_3023, data2_3023);
printList_3023(head_3023);
}
}

```

Hasil Output:

DLL Awal: 2 <-> 3 <-> 5 <->

simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->

simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->

tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->

Class ini berfungsi untuk mengimplementasikan operasi penyisipan pada *Double Linked List* melalui tiga metode utama, yaitu penambahan node di awal, akhir, dan posisi tertentu, dengan cara mengatur keterhubungan pointer ganda (next_3023 dan prev_3023) agar data dapat diakses secara dua arah.

Class ke-3 PenelusuranDLL_2511533023:

```

public class PenelusuranDLL_2511533023 {
    static void forwardTraversal_3023(NodeDLL_2511533023 head_3023) {
        NodeDLL_2511533023 curr_3023 = head_3023;
        while (curr_3023 != null) {
            System.out.print(curr_3023.data_3023 + " <-> ");
            curr_3023 = curr_3023.next_3023;
        }
        System.out.println();
    }
    static void backwardTraversal_3023(NodeDLL_2511533023 tail_3023) {
        NodeDLL_2511533023 curr_3023 = tail_3023;
        while (curr_3023 != null) {
            System.out.print(curr_3023.data_3023 + " <-> ");
            curr_3023 = curr_3023.prev_3023;
        }
        System.out.println();
    }
    public static void main(String[] args) {
        NodeDLL_2511533023 head_3023 = new NodeDLL_2511533023(1);
        NodeDLL_2511533023 second_3023 = new NodeDLL_2511533023(2);
        NodeDLL_2511533023 third_3023 = new NodeDLL_2511533023(3);

        head_3023.next_3023 = second_3023;
        second_3023.prev_3023 = head_3023;
        second_3023.next_3023 = third_3023;
        third_3023.prev_3023 = second_3023;
    }
}

```

```

        System.out.println("Penelusuran maju:");
        forwardTraversal_3023(head_3023);

        System.out.println("Penelusuran mundur:");
        backwardTraversal_3023(third_3023);

    }
}

```

Hasil Output:

Penelusuran maju:

1 <-> 2 <-> 3 <->

Penelusuran mundur:

3 <-> 2 <-> 1 <->

Class ini berfungsi untuk mendemonstrasikan kemampuan akses dua arah pada *Double Linked List* melalui metode `forwardTraversal_3023` yang menelusuri list dari depan ke belakang menggunakan pointer `next_3023`, serta metode `backwardTraversal_3023` yang menelusuri list dari belakang ke depan menggunakan pointer `prev_3023`.

Class ke-4 HapusDLL_2511533023:

```

public class HapusDLL_2511533023 {
    public static NodeDLL_2511533023 delHead_3023(NodeDLL_2511533023 head_3023)
    {
        if(head_3023 == null) {
            return null;
        }
        NodeDLL_2511533023 temp = head_3023;
        head_3023 = head_3023.next_3023;
        if(head_3023 != null) {
            head_3023.prev_3023 = null;
        }
        return head_3023;
    }
    public static NodeDLL_2511533023 delLast_3023(NodeDLL_2511533023 head_3023)
    {
        if(head_3023 == null) {
            return null;
        }
        if (head_3023.next_3023 == null) {
            return null;
        }
        NodeDLL_2511533023 curr_3023 = head_3023;
        while(curr_3023.next_3023 != null) {
            curr_3023 = curr_3023.next_3023;
        }
        if(curr_3023.prev_3023.next_3023 != null) {
            curr_3023.prev_3023.next_3023 = null;
        }
        return head_3023;
    }
}

```

```

// fungsi menghapus node posisi tertentu
public static NodeDLL_2511533023 delPos_3023(NodeDLL_2511533023 head_3023, int
pos_3023) {
    // jika DLL kosong
    if (head_3023 == null) {
        return head_3023;
    }
    NodeDLL_2511533023 curr_3023 = head_3023;
    // telusuri sampai ke node yang akan dihapus
    for (int i_3023 = 1; curr_3023 != null && i_3023 < pos_3023; ++i_3023) {
        curr_3023 = curr_3023.next_3023;
    }
    // jika posisi tidak ditemukan
    if (curr_3023 == null) {
        return head_3023;
    }
    // Update pointer
    if (curr_3023.prev_3023 != null) {
        curr_3023.prev_3023.next_3023 = curr_3023.next_3023;
    }
    if (curr_3023.next_3023 != null) {
        curr_3023.next_3023.prev_3023 = curr_3023.prev_3023;
    }
    // jika yang dihapus head
    if (head_3023 == curr_3023) {
        head_3023 = curr_3023.next_3023;
    }
    return head_3023;
}

// fungsi mencetak DLL
public static void printList_3023(NodeDLL_2511533023 head_3023) {
    NodeDLL_2511533023 curr_3023 = head_3023;
    while (curr_3023 != null) {
        System.out.print(curr_3023.data_3023 + " <-> ");
        curr_3023 = curr_3023.next_3023;
    }
    System.out.println();
}

public static void main(String[] args) {
    // buat sebuah DLL
    NodeDLL_2511533023 head_3023 = new NodeDLL_2511533023(1);
    head_3023.next_3023 = new NodeDLL_2511533023(2);
    head_3023.next_3023.prev_3023 = head_3023;
    head_3023.next_3023.next_3023 = new NodeDLL_2511533023(3);
    head_3023.next_3023.next_3023.prev_3023 = head_3023.next_3023;
    head_3023.next_3023.next_3023.next_3023 = new NodeDLL_2511533023(4);
    head_3023.next_3023.next_3023.next_3023.prev_3023 =
head_3023.next_3023.next_3023;
    head_3023.next_3023.next_3023.next_3023.next_3023 = new
NodeDLL_2511533023(5);
    head_3023.next_3023.next_3023.next_3023.next_3023.prev_3023 =
head_3023.next_3023.next_3023.next_3023;

    System.out.print("DLL Awal: ");
    printList_3023(head_3023);

    System.out.print("Setelah head dihapus: ");

```

```

    head_3023 = delHead_3023(head_3023);
    printList_3023(head_3023);

    System.out.print("Setelah node terakhir dihapus: ");
    head_3023 = delLast_3023(head_3023);
    printList_3023(head_3023);

    System.out.print("menghapus node ke 2: ");
    head_3023 = delPos_3023(head_3023, 2);
    printList_3023(head_3023);
}
}

```

Hasil Output:

DLL Awal: 1 <-> 2 <-> 3 <-> 4 <-> 5 <->

Setelah head dihapus: 2 <-> 3 <-> 4 <-> 5 <->

Setelah node terakhir dihapus: 2 <-> 3 <-> 4 <->

menghapus node ke 2: 2 <-> 4 <->

Class ini berfungsi untuk mengelola penghapusan data pada struktur *Double Linked List* melalui tiga mekanisme utama, yakni penghapusan di awal (*delHead_3023*), di akhir (*delLast_3023*), dan pada posisi tertentu (*delPos_3023*) dengan cara memanipulasi keterhubungan pointer *next_3023* dan *prev_3023* antar simpul agar struktur list tetap tersambung secara benar setelah sebuah node dihilangkan.

BAB 3

PENUTUP

3.1 Kesimpulan

Dari praktikum ini, saya dapat menyimpulkan bahwa *Double Linked List* (DLL) merupakan struktur data linier yang menawarkan fleksibilitas tinggi dalam manipulasi data melalui sistem navigasi dua arah menggunakan pointer `next_3023` dan `prev_3023`. Saya telah mempelajari dan mempraktikkan bagaimana mengelola memori secara dinamis melalui operasi penyisipan, penelusuran, dan penghapusan node pada berbagai posisi—baik di awal, di tengah, maupun di akhir—dengan tetap menjaga integritas hubungan antar simpul. Pemahaman mengenai logika pemutusan dan penyambungan alamat memori ini sangat krusial, karena meskipun DLL memerlukan ruang penyimpanan tambahan untuk pointer ekstra, kemampuan akses bolak-balik yang ditawarkannya jauh lebih efektif untuk aplikasi yang membutuhkan mobilitas data yang kompleks dan cepat.

Tugas SLL:

Class Lagu_2511533023:

```
class Lagu_2511533023 {
    private String judul_3023; // Atribut dengan akhiran NIM
    private String penyanyi_3023;
    Lagu_2511533023 next_3023; // Pointer ke lagu berikutnya
    Lagu_2511533023 prev_3023; // Pointer ke lagu sebelumnya

    // Constructor
    public Lagu_2511533023(String judul_3023, String penyanyi_3023) {
        this.judul_3023 = judul_3023;
        this.penyanyi_3023 = penyanyi_3023;
        this.next_3023 = null;
        this.prev_3023 = null;
    }

    // Getter dan Setter
    public String getJudul_3023() { return judul_3023; }
    public void setJudul_3023(String judul_3023) { this.judul_3023 = judul_3023; }

    public String getPenyanyi_3023() { return penyanyi_3023; }
    public void setPenyanyi_3023(String penyanyi_3023) { this.penyanyi_3023 =
    penyanyi_3023; }
}
```

Class ini berfungsi sebagai kerangka / blueprint yang akan dipanggil oleh class Musik_2511533023 sebagai driver.

Class kedua Musik_2511533023:

```
import java.util.Scanner;

public class Musik_2511533023 {
    private Lagu_2511533023 head_3023;
    private Lagu_2511533023 tail_3023;

    public Musik_2511533023() {
        this.head_3023 = null;
        this.tail_3023 = null;
    }

    // 1. Menambah lagu di AKHIR playlist
    public void tambahLagu_3023(String judul, String penyanyi) {
        Lagu_2511533023 laguBaru = new Lagu_2511533023(judul, penyanyi);
        if (head_3023 == null) {
            head_3023 = tail_3023 = laguBaru;
        } else {
            tail_3023.next_3023 = laguBaru;
            laguBaru.prev_3023 = tail_3023;
            tail_3023 = laguBaru;
        }
    }
}
```

```

        System.out.println("Lagu berhasil ditambahkan!");
    }

    // 2. Menghapus lagu pertama (head)
    public void hapusLaguAwal_3023() {
        if (head_3023 == null) {
            System.out.println("Playlist kosong, tidak ada lagu untuk dihapus."); //
            return;
        }
        if (head_3023 == tail_3023) {
            head_3023 = tail_3023 = null;
        } else {
            head_3023 = head_3023.next_3023;
            head_3023.prev_3023 = null;
        }
        System.out.println("Lagu pertama berhasil dihapus.");
    }

    // 3. Menampilkan semua lagu dari awal ke akhir
    public void tampilMaju_3023() {
        if (head_3023 == null) {
            System.out.println("Playlist Kosong.");
            return;
        }
        Lagu_2511533023 temp = head_3023;
        System.out.println("\n--- Playlist (Maju) ---");
        while (temp != null) {
            System.out.println(temp.getJudul_3023() + " - " +
temp.getPenyanyi_3023());
            temp = temp.next_3023;
        }
    }

    // 4. Menampilkan semua lagu dari akhir ke awal (DLL)
    public void tampilMundur_3023() {
        if (tail_3023 == null) {
            System.out.println("Playlist Kosong.");
            return;
        }
        Lagu_2511533023 temp = tail_3023;
        System.out.println("\n--- Playlist (Mundur) ---");
        while (temp != null) {
            System.out.println(temp.getJudul_3023() + " - " +
temp.getPenyanyi_3023());
            temp = temp.prev_3023;
        }
    }

    // 5. Mencari lagu berdasarkan judul (case-insensitive)
    public void cariLagu_3023(String judul) {
        Lagu_2511533023 temp = head_3023;
        boolean ditemukan = false;
        while (temp != null) {
            if (temp.getJudul_3023().equalsIgnoreCase(judul)) {
                System.out.println("Lagu ditemukan: " + temp.getJudul_3023() + " |
Penyanyi: " + temp.getPenyanyi_3023());
                ditemukan = true;
                break;
            }
        }
    }

```

```

        temp = temp.next_3023;
    }
    if (!ditemukan) System.out.println("Lagu '" + judul + "' tidak ditemukan.");
}

// Main method untuk menjalankan menu
public static void main(String[] args) {
    Musik_2511533023 playlist = new Musik_2511533023();
    Scanner input = new Scanner(System.in);
    int pilihan;

    do {
        System.out.println("\n=== Playlist Musik NIM: 2511533023 ===");
        System.out.println("1. Tambah Lagu");
        System.out.println("2. Hapus Lagu Pertama");
        System.out.println("3. Lihat Playlist (Maju)");
        System.out.println("4. Lihat Playlist (Mundur)");
        System.out.println("5. Cari Lagu");
        System.out.println("6. Keluar");
        System.out.print("Pilihan: ");
        pilihan = input.nextInt();
        input.nextLine(); // consume newline

        switch (pilihan) {
            case 1:
                System.out.print("Judul: ");
                String judul = input.nextLine();
                System.out.print("Penyanyi: ");
                String penyanyi = input.nextLine();
                playlist.tambahLagu_3023(judul, penyanyi);
                break;
            case 2:
                playlist.hapusLaguAwal_3023();
                break;
            case 3:
                playlist.tampilMaju_3023();
                break;
            case 4:
                playlist.tampilMundur_3023();
                break;
            case 5:
                System.out.print("Masukkan Judul yang dicari: ");
                String cari = input.nextLine();
                playlist.cariLagu_3023(cari);
                break;
            case 6:
                System.out.println("Keluar program...");
                break;
            default:
                System.out.println("Pilihan tidak valid.");
        }
    } while (pilihan != 6);
    input.close();
}
}

```


Hasil Output:

```
=== Playlist Musik NIM: 2511533023 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: Drunk Text
Penyanyi: Henry Moodie
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511533023 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: Glimpse of us
Penyanyi: Joji
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511533023 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Masukkan Judul yang dicari: Drunk Text
Lagu ditemukan: Drunk Text | Penyanyi: Henry Moodie

=== Playlist Musik NIM: 2511533023 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3

--- Playlist (Maju) ---
Drunk Text - Henry Moodie
Glimpse of us - Joji

=== Playlist Musik NIM: 2511533023 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 4
```

```

--- Playlist (Mundur) ---
Glimpse of us - Joji
Drunk Text - Henry Moodie

=== Playlist Musik NIM: 2511533023 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Lagu pertama berhasil dihapus.

=== Playlist Musik NIM: 2511533023 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3

--- Playlist (Maju) ---
Glimpse of us - Joji

=== Playlist Musik NIM: 2511533023 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Masukkan Judul yang dicari: Drunk Text
Lagu 'Drunk Text' tidak ditemukan.

=== Playlist Musik NIM: 2511533023 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 6
Keluar program...

```

Class ini berfungsi sebagai pengelola utama (*manager class*) yang mengimplementasikan struktur data **Double Linked List** untuk mengatur operasional playlist lagu secara dinamis. Kelas ini bertanggung jawab mengontrol pointer `head_3023` dan `tail_3023` agar setiap lagu (node) dapat terhubung dua arah, memungkinkan pengguna untuk menambahkan lagu di akhir, menghapus lagu pertama, serta melakukan penelusuran playlist baik secara maju maupun mundur. Selain sebagai pusat logika manipulasi data, kelas ini juga menyediakan antarmuka menu interaktif bagi pengguna untuk berinteraksi dengan sistem playlist tersebut.